Type Systems That Reinterpret Themselves

ANONYMOUS AUTHOR(S)

We advocate that type systems should be able to reinterpret their own terms in any internal model of themselves: equipped with a single tool of reinterpretation, you can express many desired features of type systems, including universe polymorphism, limited effect polymorphism, external parametricity and more. To this end, we present two type systems capable of self-reinterpretation, namely STLC-R and MLTT-R, prove their basic properties and show how the features we claim they contain are definable on the library level.

1 Introduction

Consider the following well-known function which maps over elements of the list, written in Agda:

```
map : ∀ {A B : Set} → (A → B) → List A → List B
map f [] = []
map f (x ∷ xs) = f x ∷ map f xs
```

Enjoying both composability of this function and simplicity of its type, we would like to reuse its definition as much as possible. Up to a certain level, we would surely be able to do so: after all, its definition is polymorphic in source and target types, so we can instantiate it with whatever types we want. But we might wish for much more; of things that come to our mind are the following:

- Reuse `map` at higher universe levels, e.g., given a *type operator*, map over *list of types*;
- Automagically obtain a proof of the free theorem for `map` – that is, some term of type

$$\forall A\, A'\, (Q : A \to A' \to \mathtt{Prop}), \forall B\, B'\, (R : B \to B' \to \mathtt{Prop}),$$
$$\forall f\, f', (\forall x\, x', Q\, x\, x' \to R(f\, x)\, (f'\, x')) \to$$
$$\forall (\mathtt{xs} : \mathtt{List}\, A)(\mathtt{xs}' : \mathtt{List}\, A'), Q^*\, \mathtt{xs}\, \mathtt{xs}' \to R^*(\mathtt{map}\, f\, \mathtt{xs})(\mathtt{map}\, f'\, \mathtt{xs}');$$

- Given a group G, sets A and B with actions a, b of G on them, and a function $f : A \to B$ which respects a and b, prove that `map` f respects actions on `List` A and `List` B induced by a and b, respectfully;..

The first wish is, of course, granted by adjusting the definition and making it level-polymorphic. Second one is possible in a system with *internal parametricity*, as exemplified by Altenkirch and others [1]. The last wish, however, has to be fulfilled manually, albeit with a simple list induction.

A shift in perspective might help. Instead of tweaking the definition of `map`, let's remember all of the metatheoretic power our type system already has. Parametricity is a first hint; another is due to Lambek [7] who established that simply-typed lambda calculus (STLC) is an *internal language* of *cartesian closed categories* (CCC) [1]. In particular, any term written in STLC can be interpreted in any cartesian closed category; this way, you don't have to repeat your construction in any CCC you meet, you just reinterpret the STLC term you already have [5]. This correspondence can be extended further into dependently typed systems, establishing correspondence with certain kinds of topoi and enabling us to do synthetic mathematics: prove general statements in higher-order intuitionistic logic and translate them into particular theorems about our topos of interest [10].

All of this power, unfortunately, lies beyond what usual type systems can see, since that's their metatheoretical properties. But what if they were internalized? What if we had a way to *reinterpret terms of a type system inside itself*? We experiment with this idea and end up with two minimal systems, STLC-R (section 2) and MLTT-R (section 3), based on simply typed lambda calculus and Martin-Löf type theory respectively. Turns out that their capability of *self-reinterpretation* is expressive enough to cover all of the discussed usecases, including restricted form of effect

[1] This is, of course, part of well-known Curry-Howard-Lambek correspondence.

polymorphism (section 2.2), synthetic mathematics (section 3.2.3), universe polymorphism (section 3.2.1), parametricity (section 3.2.2) and more.

1.1 Contributions

- We start by giving the syntax and semantics of our first type system that reinterprets itself. It is based on simply typed lambda calculus and is called STLC-R. We prove that STLC-R is sound, and that global type inference is decidable (section 2).
- As expected, you can reinterpret STLC-R terms in any cartesian closed category you define inside it, including categories of differentiable manifolds for applications in autodifferentiation, Kleisli categories of certain monads and more (section 2.2).
- Building upon the insights gathered from developing STLC-R, we give the syntax and semantics of a dependently typed self-reinterpreting language based on Martin-Löf type theory called MLTT-R. We prove soundness of MLTT-R as well, and motivate bidirectional typing discipline for it (section 3).
- As we intended, MLTT-R incorporates both a restricted form of universe polymorphism and external parametricity by virtue of reinterpretation only. And, of course, you can use MLTT-R as a system for synthetic mathematics with internal reuse of synthetic results (section 3.2).

2 Almost simply typed reinterpretable language: STLC-R

We start with a simpler language to iron out the details and actually figure out how to properly reinterpret terms internally before moving on to dependently typed system. Our starting point is simply typed lambda calculus with products, which we call «STLC» throughout this paper. As discussed before, we will add self-reinterpretation capabilities to it to enable reinterpretation in any «model» of STLC specifiable inside our language. Here's the thing we *wish* to do with it, where $t \mathbin{@} \mathcal{M}$ is an operator of reinterpretation of term t in internal model $\mathcal{M}$:

$$(\lambda f x y.(f x, y)) \mathbin{@} \langle (\circ), \mathtt{pair}, \mathtt{proj}_1, \mathtt{proj}_2, \mathtt{lam}, \mathtt{eval}, \ldots \rangle =$$
$$= \mathtt{lam}\,(\mathtt{lam}\,(\mathtt{lam}\,(\mathtt{pair}\,(\mathtt{eval} \circ \mathtt{pair}\,(\mathtt{proj}_2 \circ \mathtt{proj}_1 \circ \mathtt{proj}_1)\,(\mathtt{proj}_2 \circ \mathtt{proj}_1))\,\mathtt{proj}_2)))$$

Where `lam`, `pair` and `eval` are correspondingly constructor of an exponential object, constructor of a product and evaluation morphism, in the same way they are specified in the definition of cartesian closed categories.

There are a few things to ponder:

- How to reinterpret free variables?
- How to type universes?
- We're in a simply typed setting, so there's no way to enforce that universe satisfies coherence conditions on their operations. How to structure computation in such a way that it doesn't break Church-Rosser property?

To answer the first question, we actually disallow (most of) free variables in a terms that are to be reinterpreted since it is not clear how to lift arbitrary *value* into current universe, so we have to enforce some kind of a typing discipline for introducing terms amenable to reinterpretation. In a sense, reinterpretation is a strong form of polymorphism, so let's structure our grammar and typing rules similarly to how it's done in Hindley-Milner type systems today and see how far we can go; that's figure 1.

In figure 2, we mostly repeat usual declarative typing rules of a Hindley-Milner type system [4] (TODO: check source) with an addition of a rule for reinterpretation (discussed below) and an important change in `let` case: as discussed, we don't take local context in account when checking

MonoType	S, T, C, P, E, TT	$::=$	α	type variable
		$\mid$	$\top$	unit type
		$\mid$	$S \times T$	product type
		$\mid$	$S \to T$	arrow type
PolyType	σ	$::=$	T	monomorphic type
		$\mid$	$\forall \alpha.\sigma$	forall type
Term	s, t	$::=$	x	variable
		$\mid$	$()$	singleton
		$\mid$	(s, t)	pair
		$\mid$	$\pi_1\, t$	first projection
		$\mid$	$\pi_2\, t$	second projection
		$\mid$	$\lambda x.t$	abstraction
		$\mid$	$t\, s$	application
		$\mid$	$\texttt{let}\, x = s\, \texttt{in}\, t$	polymorphic binding
		$\mid$	$x^{\mathcal{M}}$	reinterpretation
Model	$\mathcal{M}$	$::=$	$\langle T, t, \ldots \rangle$	model def. (see below)

Fig. 1. STLC-R grammar

$$\frac{x : \sigma \in \Gamma}{\Gamma \vdash x : \sigma} \qquad \Gamma \vdash () : \top \qquad \frac{\Gamma \vdash s : S \qquad \Gamma \vdash t : T}{\Gamma \vdash (s, t) : S \times T} \qquad \frac{\Gamma \vdash t : S \times T}{\Gamma \vdash \pi_1\, t : S} \qquad \frac{\Gamma \vdash t : S \times T}{\Gamma \vdash \pi_2\, t : T}$$

$$\frac{\Gamma, x : S \vdash t : T}{\Gamma \vdash \lambda x.t : S \to T} \qquad \frac{\Gamma \vdash t : S \to T \qquad \Gamma \vdash s : S}{\Gamma \vdash t\, s : T} \qquad \frac{\Gamma \vdash t : \sigma}{\Gamma \vdash t : \forall \alpha.\sigma} \qquad \frac{\Gamma \vdash t : \forall \alpha.\sigma}{\Gamma \vdash t : \sigma[\alpha \coloneqq T]}$$

$$\frac{\mathrm{fv}(\sigma) = \varnothing \qquad \forall \Gamma \vdash s : \sigma \qquad \forall \Gamma, x : \sigma, \Delta \vdash t : T}{\forall \Gamma, \Delta \vdash \texttt{let}\, x = s\, \texttt{in}\, t : T} \qquad \frac{x : \forall \vec{\alpha}.T \in \forall \Gamma \qquad \forall \Gamma, \Delta \vdash \mathcal{M}\ \text{model}}{\forall \Gamma, \Delta \vdash x^{\mathcal{M}} : [\![T]\!]^{\mathcal{M}}_{\left[\vec{\alpha} \coloneqq \vec{S}\right]}}$$

$$\frac{\begin{array}{c}\mathrm{fv}(C) \cup \mathrm{fv}(P) \cup \mathrm{fv}(E) \subseteq \{0, 1\} \\ \Gamma \vdash \texttt{id} : \forall \alpha.C[\alpha, \alpha] \qquad \Gamma \vdash \texttt{comp} : \forall \alpha\beta\gamma.C[\beta, \gamma] \to C[\alpha, \beta] \to C[\alpha, \gamma] \\ \mathrm{fv}(TT) = \varnothing \qquad \Gamma \vdash \texttt{cnst} : \forall \alpha.C[\alpha, TT] \qquad \Gamma \vdash \texttt{pair} : \forall \alpha\beta\gamma.C[\gamma, \alpha] \to C[\gamma, \beta] \to C[\gamma, P[\alpha, \beta]] \\ \Gamma \vdash \texttt{proj}_1 : \forall \alpha\beta.C[P[\alpha, \beta], \alpha] \qquad \Gamma \vdash \texttt{proj}_2 : \forall \alpha\beta.C[P[\alpha, \beta], \beta] \\ \Gamma \vdash \texttt{lam} : \forall \alpha\beta\gamma.C[P[\gamma, \beta], \alpha] \to C[\gamma, E[\alpha, \beta]] \qquad \Gamma \vdash \texttt{eval} : \forall \alpha\beta.C[P[E[\alpha, \beta], \beta], \alpha]\end{array}}{\Gamma \vdash \langle \Lambda 01.C, \texttt{id}, \texttt{comp}, TT, \texttt{cnst}, \Lambda 01.P, \texttt{pair}, \texttt{proj}_1, \texttt{proj}_2, \Lambda 01.E, \texttt{lam}, \texttt{eval} \rangle\ \text{model}}$$

Fig. 2. STLC-R declarative typing rules

polymorphic term s. Instead, it can only reference other polymorphic terms. This is safe since other polymorphic terms are constrained in the same way, and can be safely used inside s.

To check models, we abuse our type system a bit in order to avoid adding type operators and complicate the type system: types of arrows C, products P and exponentials E are specified as open monomorphic types which can use two type variables, 0 and 1. This fact is reflected in the syntax of a model using notation $\Lambda 01.C$, $\Lambda 01.P$ and $\Lambda 01.E$ respectively. 0 and 1 are then filled

$$u, v ::= n \mid () \mid (u, v) \mid \lambda x.v$$
$$n ::= x \mid \pi_i\, n \mid n\, v$$

Fig. 3. Values and neutral terms of STLC-R

$$\frac{s \longrightarrow s'}{(s,t) \longrightarrow (s',t)} \qquad \frac{t \longrightarrow t'}{(s,t) \longrightarrow (s,t')} \qquad \frac{t \longrightarrow t'}{\pi_i t \longrightarrow \pi_i t'} \qquad \pi_i(t_1,t_2) \longrightarrow t_i$$

$$\frac{t \longrightarrow t'}{\lambda x.t \longrightarrow \lambda x.t'} \qquad \frac{s \longrightarrow s'}{s\,t \longrightarrow s'\,t} \qquad \frac{t \longrightarrow t'}{s\,t \longrightarrow s\,t'} \qquad (\lambda x.t)s \longrightarrow t[x := s]$$

$$\frac{s \longrightarrow s'}{\mathtt{let}\, x = s\, \mathtt{in}\, t \longrightarrow \mathtt{let}\, x = s'\, \mathtt{in}\, t} \qquad \frac{t \longrightarrow t'}{\mathtt{let}\, x = s\, \mathtt{in}\, t \longrightarrow \mathtt{let}\, x = s\, \mathtt{in}\, t'}$$

$$\frac{\mathrm{fv}(v) = \varnothing}{\mathtt{let}\, x = v\, \mathtt{in}\, t \longrightarrow t[\![x := v]\!]}$$

Fig. 4. STLC-R computation rules

in by substitution when we need to type model operations (notation $T[S_1, S_2]$ is a shorthand for $T[0 := S_1, 1 := S_2]$).

We could generalize model definition to include something else as objects instead of types, but there is no way to build meaningful C,P and E with anything other than types in base system.

Definition 2.1. Let $\mathcal{M} = \langle \Lambda 01.C, \mathtt{id}, \mathtt{comp}, TT, \mathtt{cnst}, \Lambda 01.P, \mathtt{pair}, \mathtt{proj}_1, \mathtt{proj}_2, \Lambda 01.E, \mathtt{lam}, \mathtt{eval} \rangle$. Then, interpretation of type in a model is the following function:

$$[\![T]\!]^{\mathcal{M}}_{\sigma} = C[TT, T^{\mathcal{M}}_{\sigma}] \tag{1}$$
$$\alpha^{\mathcal{M}}_{\sigma} = \sigma\alpha \tag{2}$$
$$\top^{\mathcal{M}}_{\sigma} = TT \tag{3}$$
$$(S \times T)^{\mathcal{M}}_{\sigma} = P[S^{\mathcal{M}}_{\sigma}, T^{\mathcal{M}}_{\sigma}] \tag{4}$$
$$(S \longrightarrow T)^{\mathcal{M}}_{\sigma} = E[S^{\mathcal{M}}_{\sigma}, T^{\mathcal{M}}_{\sigma}]. \tag{5}$$

Having defined typing rules, let's pin down computation rules in order to prove some useful properties. General idea is the following: before inlining a `let` binding, we first have to make sure that the polymorphic term is closed and is in normal form; in all other cases, standard reductions of STLC apply.

Definition 2.2. By $t[\![x \coloneqq v]\!]$ we mean an operation which works as a regular substitution of v into t except for the case of reinterpretation:

$$x^{\mathcal{M}}[\![x \coloneqq v]\!] = [\![v]\!]^{\mathcal{M}}/\varepsilon \tag{6}$$

$$[\![()]\!]^{\mathcal{M}}/\gamma = \mathtt{cnst} \tag{7}$$

$$[\![(s,t)]\!]^{\mathcal{M}}/\gamma = \mathtt{pair}\,[\![s]\!]^{\mathcal{M}}/\gamma\,[\![t]\!]^{\mathcal{M}}/\gamma \tag{8}$$

$$[\![\pi_i\,t]\!]^{\mathcal{M}}/\gamma = \mathtt{comp}\,\mathtt{proj}_i\,[\![t]\!]^{\mathcal{M}}/\gamma \tag{9}$$

$$[\![x]\!]^{\mathcal{M}}/\gamma = \gamma\,x \tag{10}$$

$$[\![\lambda x.t]\!]^{\mathcal{M}}/\gamma = \mathtt{lam}\,[\![t]\!]^{\mathcal{M}}/[\vec{y} \coloneqq \mathtt{comp}\,\mathtt{proj}_1(\gamma\,\vec{y}),\ x \coloneqq \mathtt{proj}_2] \tag{11}$$

$$[\![t\,s]\!]^{\mathcal{M}}/\gamma = \mathtt{comp}\,\mathtt{eval}\,\left(\mathtt{pair}\,[\![t]\!]^{\mathcal{M}}/\gamma\,[\![s]\!]^{\mathcal{M}}/\gamma\right) \tag{12}$$

where model operations $\mathtt{cnst}, \mathtt{pair}$ and others are taken from $\mathcal{M}$ and ε is an empty substitution.

2.1 Metatheoretical properties of STLC-R

Here we only study some basic properties of a system to make sure that everything works as expected: computations don't diverge; types are preserved by all computations; all of the types can be inferred if you wish to do so; everything terminates and, modulo reduction of `let`-bindings, this is the same old STLC we started with. Proofs are mostly following standard techniques, since rules for reinterpretation are structured to directly plug into proofs of properties of STLC.

Theorem 2.3 (Church-Rosser theorem for STLC-R). *Reflexive transitive closure of* $(\rightarrow)$ *has Church-Rosser property.*

Proof. Classic proof via parallel reduction relation [12]. Parallel reduction is defined as:

$$t \twoheadrightarrow t \qquad \frac{s \twoheadrightarrow s' \qquad t \twoheadrightarrow t'}{(s,t) \twoheadrightarrow (s',t')} \qquad \frac{t \twoheadrightarrow t'}{\pi_i\,t \twoheadrightarrow \pi_i\,t'} \qquad \frac{t_1 \twoheadrightarrow t_1' \qquad t_2 \twoheadrightarrow t_2'}{\pi_i(t_1,t_2) \twoheadrightarrow t_i'}$$

$$\frac{t \twoheadrightarrow t'}{\lambda x.t \twoheadrightarrow \lambda x.t'} \qquad \frac{s \twoheadrightarrow s' \qquad t \twoheadrightarrow t'}{s\,t \twoheadrightarrow s'\,t'} \qquad \frac{s \twoheadrightarrow s' \qquad t \twoheadrightarrow t'}{(\lambda x.s)\,t \twoheadrightarrow s'[x \coloneqq t']}$$

$$\frac{s \twoheadrightarrow s' \qquad t \twoheadrightarrow t'}{\mathtt{let}\,x = s\,\mathtt{in}\,t \twoheadrightarrow \mathtt{let}\,x = s'\,\mathtt{in}\,t'} \qquad \frac{t \twoheadrightarrow t'}{\mathtt{let}\,x = v\,\mathtt{in}\,t \twoheadrightarrow t'[\![x \coloneqq v]\!]}$$

□

Theorem 2.4 (Type preservation for STLC-R). *The following is true:*

(1) *If* $\mathcal{M}$ model, $\gamma\,x : C[\Gamma^{\mathcal{M}}, \Gamma(x)^{\mathcal{M}}]$ *for all* $x \in \Gamma$ *and* $\Gamma \vdash t : T$, *then* $[\![t]\!]^{\mathcal{M}}/\gamma : C[\Gamma^{\mathcal{M}}, T^{\mathcal{M}}]$.
(2) *If* $\Gamma \vdash s : S$ *and* $\Gamma, x : S, \Delta \vdash t : T$, *then* $\Gamma, \Delta \vdash t[x \coloneqq s] : T$.
(3) *If* $\Gamma \vdash v : \sigma$ *and* $\Gamma, x : \sigma, \Delta \vdash t : T$, *then* $\Gamma, \Delta \vdash t[\![x \coloneqq v]\!] : T$.
(4) *If* $\Gamma \vdash t : T$ *and* $t \rightarrow t'$, *then* $\Gamma \vdash t' : T$.

Proof. By structural induction on terms and $(\rightarrow)$. □

Theorem 2.5 (Strong normalization for STLC-R). *Closed terms typable in STLC-R are strongly normalizing with respect to* $(\rightarrow)$.

Proof. By Tait's method of logical relations. Let SN be a set of strongly normalizing terms. Let there be the following relation $\mathcal{R} \subseteq \mathsf{PolyType} \times \mathsf{Term}$:

- $t \in \mathcal{R}_{\alpha}, \mathcal{R}_{\top}$ iff $t \in SN$;
- $t \in \mathcal{R}_{T_1 \times T_2}$ iff $\pi_i\, t \in \mathcal{R}_{T_i}$ for $i = 1, 2$;
- $t \in \mathcal{R}_{S \to T}$ iff for all $s \in \mathcal{R}_S$, $t\, s \in \mathcal{R}_T$;
- $t \in \mathcal{R}_{\forall \vec{\alpha}.T}$ iff $t \in SN_{\texttt{let}}$ and for all $\mathcal{U} \in \mathcal{R}_{univ}$ and $\vec{S}$, $nf_{\texttt{let}}(t) \mathbin{@} \mathcal{U} \in \mathcal{R}_{\mathcal{U}[\![T]\!][\vec{\alpha} := \vec{S}]}$.

$\mathcal{U} \in \mathcal{R}_{univ}$ is defined similarly by requiring that all terms inside $\mathcal{U}$ belong to respective $\mathcal{R}$-s. Then we can prove our main lemma by mutual induction on typing rules:

- If $\Gamma \vdash t : T$ and γ is a substitution s.t. $\gamma\, x \in \mathcal{R}_{\Gamma(x)}$ for all $x \in \Gamma$ then $\gamma\, t \in \mathcal{R}_T$.
- If $\forall \Gamma \vdash t : T$, $\vec{\alpha} = fv(T)$ and γ is a substitution s.t. $\gamma\, x \in \mathcal{R}_{\Gamma(x)}$ for all $x \in \Gamma$ then $\gamma\, t \in \mathcal{R}_{\forall \vec{\alpha}.T}$.

Noticing $\mathcal{R}_T \subseteq SN$ completes the proof. □

Theorem 2.6 (Conservativity over STLC). *Normal forms of closed terms typable in STLC-R are typable in STLC.*

Proof. Thanks to type preservation, normal form $nf(t)$ is typable in STLC-R. Normal form cannot contain let-bindings by definition of $(\to)$, therefore each subterm is $\forall$-closed. Therefore $nf(t)$ has no reinterpretation subterms. □

Theorem 2.7. *Global type inference for STLC-R is decidable.*

Proof. It is enough to extend any unification-based type inference algorithm for Hindley-Milner type system to support universes and reinterpretation.

- Since universes are provided explicitly, it is enough to *check* them.
- Since polymorphic functions do not depend on local context, their type is fully resolved when it's added to context.
- Once universe is checked in $x \mathbin{@} \mathcal{U}$, we can directly compute $\mathcal{U}[\![T]\!]$ (since $\mathcal{U}$ is known and T is fully resolved) and instantiate it with fresh metavariables in $\vec{S}$.

□

2.2 Useful reinterpretations inside STLC-R

The idea to interpret STLC terms in cartesian closed categories is really not new and is already thoroughly explored by Conal Elliott in "Compiling to Categories" [5]; our addition here is that we internalized reinterpretation inside the calculus without working through compiler plugin. So all of the main applications from the cited paper are still applicable here whenever they are expressible as universes in STLC-R. As STLC-R is not the main focus of the paper, we will not be reiterating the examples here, but we would like to add just one more example.

Following Moggi [9], we can use monads to model side effects in a language. What is new here is that instead of parameterizing a function by arbitrary monad, we can simply *reinterpret* pure terms as if they were run in monadic context. Therefore, our universe is a Kleisli category; however, it is only cartesian, but in general not cartesian *closed*. The problem is the following operation (here F is a monad in question, E is whatever exponential object we were to use):

$$\texttt{lam} : (\alpha \times \beta \to F[\gamma]) \to (\alpha \to F[E[\gamma, \beta]])$$

In other words, F has to *distribute* over an arrow. Well, at least arrow type distributes over itself:

$$\begin{aligned}
\mathcal{U}_E = \langle & \\
& \Lambda 01.\, 0 \to E \to 1, \\
& \lambda xe.x, \\
& \lambda gfxe.g(fxe)e, \\
& \top, \\
& \lambda e.(), \\
& \Lambda 01.\, 0 \times 1, \\
& \lambda fgxe.(fxe, gxe), \\
& \lambda pe.\pi_1\, p, \\
& \lambda pe.\pi_2\, p, \\
& \Lambda 01.\, 0 \to 1, \\
& \lambda fxey.f(x,y)e, \\
& \lambda pe.(\pi_1\, p)\, (\pi_2\, p) \\
\rangle &
\end{aligned}$$

Interestingly enough, if, using Curry-Howard correspondence, we take closed STLC term t to represent proof of an intuitionistic proposition, $t\, @\, \mathcal{U}_E$ corresponds to hypothesis elimination in intuitionistic logic.

We *could* write down a universe for arbitrary monad if we had capability to interpret terms which do not use abstractions in arbitrary cartesian category, which is actually achievable with [5]. We leave study of such "partial" interpretation for future work.

3 Dependently typed reinterpretable language: MLTT-R

Now we are ready to take on the main challenge: design a minimal dependently typed reinterpretable language, and provide internal models of itself to yield universe polymorphism, external parametricity and do synthetic mathematics inside it. As a starting point, we take Martin-Lof type theory with $\Pi-,\Sigma-,\mathrm{Id}-$ types and infinite cumulative hierarchy of universes. The problems are similar to the ones we had with simply-typed calculus:

- How to reinterpret free variables?
- How to type universes?
- Now, there is a way to enforce that internal model satisfies coherence conditions on their operations. How to structure computation in such a way that it doesn't break Church-Rosser property?
- How to respect conversion rule?

As before, we sidestep the question with free variables by considering a term for reinterpretation iff it is closed in local context. To properly capture this property, we introduce a modality of closed terms $\Box$ which is already extensively studied in previous works (TODO: provide refs). To type universes, we have to consider:

- Typing rules for MLTT are necessarily mutually recursive with rules of context well-formedness. Which framework to use to capture this in a model?
- How to provide a "universe" for each level in the interpreted term?

To deal with the first problem, we use Categories-with-Families [3] as one of the most convenient models of dependent type theories in use today. Second one, however, is reminiscent of classical

circularity paradoxes in logic and computer science: if we were to provide a model to each universe at each finite level, in which universe does the model lie? The first solution which comes to mind is to add another sequence of levels $\omega, \omega + 1, \ldots$ and allow to use only finite-level universes inside reinterpretable term; such a system, however, would not "eat itself", a property that we wanted to achieve. Consider instead the following observation: in a system without level binders, each term uses only a finite number of levels. Hence to reinterpret this term, we need to only provide a finite number of universe models, which can together fit into some universe of a finite level.

The problem with Church-Rosser is solved in absolutely the same way as with simply-typed case: we just do not perform any β-reductions inside a let-binding, only inline `let` declarations before performing a substitution.

The problem with conversion rule is the following: in definition of a universe, we can now ask for propositional proofs of coherence. Thus types which were *definitionally* equal before reinterpretation are now only *propositionally* equal. To solve this issue, we start by first pinpointing the locations where conversion rule is actually used; in order to do this, we *need* to use bidirectional type checking. After translation, point of conversion is wrapped in a `rewrite` call.

But how do we synthesize a propositional proof of equivalence? Here we translate a chain of reductions performed during checking of definitional equality into a chain of propositional equivalences taken from coherence conditions proven in the definition of a model.

To actually compute interpretation in MLTT-R, we need two mutually recursive functions: $t \mathbin{@}^{\gamma}_{\Leftarrow T} \mathcal{U}$ and $t \mathbin{@}^{\gamma}_{\Rightarrow} \mathcal{U}$ for checkable and synthesizing terms, respectively. **We're sorry to be glossing over details here and will provide all definitions in the updated version of the paper.**

3.1 Metatheoretical properties of MLTT-R

Once again, we only try to make sure of the most essential properties: that computations do not diverge, that types during computations are preserved, all computations terminate and that resulting type system is, in essence, the same as base MLTT modulo inlining of `let`-bindings.

Theorem 3.1 (Church-Rosser theorem for STLC-R). *Reflexive transitive closure of* $(\rightarrow)$ *has Church-Rosser property.*

Proof. Classic proof via parallel reduction relation [12]. Both reductions outside and inside `let`-reduction have to be parallelized in order to prove the theorem. □

Theorem 3.2 (Type preservation for MLTT-R).

Proof. By structural induction on terms and $(\rightarrow)$. It is important that, due to the way computations are structured, $t \triangleleft_x s$ is always applied to closed s, therefore $t@\mathcal{U}$ is always applied to t that is $\Box$-closed. □

Theorem 3.3 (Strong normalization for MLTT-R). *Closed terms typable in MLTT-R are strongly normalizing with respect to* $(\rightarrow)$.

Proof. Tait's method of logical relations for MLTT with modifications similar to ones performed in a proof of strong normalization for STLC-R should suffice. However, we're not completely sure. □

Theorem 3.4 (Conservativity over MLTT). *Normal forms of closed terms typable in MLTT-R are typable in MLTT.*

Proof. Thanks to type preservation, normal form $nf(t)$ is typable in MLTT-R. Normal form cannot contain let-bindings by definition of $(\rightarrow)$, therefore each subterm has no $\Box$ in context. Therefore $nf(t)$ has no reinterpretation subterms. □

Context	Γ	::=	$\cdot$	empty context
		|	$\Gamma, x : T$	variable extension
		|	$\Gamma.\,🔒$	context locking
Type	T, S	::=	$\mathcal{U}_\ell$	universe
		|	$\mathrm{El}_\ell(t)$	universal type family
		|	Nat	natural number type
		|	$(x : S) \times T$	Σ-type
		|	$(x : S) \to T$	Π-type
		|	$t_1 =_T t_2$	identity type
		|	$\Box\, T$	type of closed terms
		|	$\mathcal{M}_{\ell_1,\ldots,\ell_\kappa}$	type of models
Level	ℓ, κ, j	$\in$	$\mathbb{N}$	
Term	t, s	::=	$x \mid y \mid \mathtt{m} \mid \ldots$	variables
		|	U_ℓ	code of a universe
		|	$\mathtt{nat}_{\mathcal{U}}$	code of a natural number type
		|	0	zero
		|	$\mathrm{S}\, t$	successor
		|	$\mathrm{natind}_{x.T}\, t\, \{s_0, y \mapsto s_S\}$	induction
		|	$(x : s) \times_{\mathcal{U}} t$	code of a Σ-type
		|	(s, t)	pairing
		|	$\pi_1\, t \mid \pi_2\, t$	projections
		|	$(x : s) \to_{\mathcal{U}} t$	code of a Π-type
		|	$\lambda x.\, t$	abstraction
		|	$t\, s$	application
		|	$\mathtt{id}_{\mathcal{U}}\, t\, t_1\, t_2$	code of an identity type
		|	refl	reflexivity
		|	$\mathrm{J}_{xy.S}\, t\, s$	J rule (elimination of equality type)
		|	$\Box_{\mathcal{U}}\, t$	code of a box type
		|	$[t]_🔒$	locking
		|	$[t]_{🔓}$	unlocking
		|	$[\![t]\!]^{\ell_0,\ldots,\ell_\kappa}$	internal interpretation
		|	$t : T$	ascription

Fig. 5. MLTT-R grammar

3.2 Useful reinterpretations inside MLTT-R

Here we list some of the language features which reinterpretation enables without any additional language support.

3.2.1 Crude, but effective stratification. Due to McBride [8], a technique of crude, but effective stratification works as follows: instead of providing complete support for level polymorphism, let's consider a *displacement operator* which, given a term t using universes $U_i, U_j, U_k, \ldots$, produces a term which is equal to t except for universes which are replaced with $U_{i+1}, U_{j+1}, U_{k+1}, \ldots$. The displaced term is also well-typed, however it is defined «one level higher». Notice that displacement operator is *definable* in MLTT as $_ @ \mathcal{U}_{\text{McBride}}$ where

$$\mathcal{U}_{\text{McBride}} = \langle \Lambda l. U_{suc\, l}, \ldots \rangle$$

$$\textsc{WF-Nil}\ \frac{}{\cdot\ \text{wf}} \qquad \textsc{WF-Var}\ \frac{\Gamma\ \text{wf} \qquad \Gamma \vdash \text{level}(T) = \ell}{\Gamma, x : T\ \text{wf}} \qquad \textsc{WF-Lock}\ \frac{\Gamma\ \text{wf}}{\Gamma.🔒\ \text{wf}}$$

$$\textsc{Level-Uni}\ \frac{\Gamma\ \text{wf}}{\Gamma \vdash \text{level}(\mathcal{U}_\ell) = \ell + 1} \qquad \textsc{Level-El}\ \frac{\Gamma \vdash t \Leftarrow \mathcal{U}_\ell}{\Gamma \vdash \text{level}(\text{El}_\ell(t)) = \ell} \qquad \textsc{Level-Nat}\ \frac{\Gamma\ \text{wf}}{\Gamma \vdash \text{level}(\text{Nat}) = 0} \qquad \textsc{Level-Box}\ \frac{\Gamma \vdash \text{level}(T) = \ell}{\Gamma \vdash \text{level}(\Box\, T) = \ell}$$

$$\textsc{Level-Sigma, Level-Pi}\ \frac{\Gamma \vdash \text{level}(S) = \kappa \qquad \Gamma, x : S \vdash \text{level}(T) = \ell}{\Gamma \vdash \text{level}((x : S) \times T) = \text{level}((x : S) \to T) = \max(\kappa, \ell)}$$

$$\textsc{Level-Id}\ \frac{\Gamma \vdash \text{level}(T) = \ell \qquad \Gamma \vdash t_1, t_2 \Leftarrow T}{\Gamma \vdash \text{level}(t_1 =_T t_2) = \ell}$$

$$\textsc{Level-Model}\ \frac{\Gamma\ \text{wf}}{\Gamma \vdash \text{level}\left(\mathcal{M}_{\ell_0,\ldots,\ell_\kappa}\right) = \max(\ell_0, \ldots, \ell_\kappa) + 1}$$

Fig. 6. Context well-formedness & level placement rules

3.2.2 Internal external parametricity. Demonstrated by Reynolds [11], parametricity is a property of a type system that, in simple words, «polymorphic function cannot meaningfully depend on generic inputs». Thus any polymorphic function of type σ satisfies a certain «free theorem» $[\![\sigma]\!]$ defined via relational interpretation of a type.

In recent works [2] [1], a streamlined definition is given for parametricity interpretation of dependently-typed systems dubbed «external parametricity». Once again, this relational interpretation is definable in our system *internally* with the help of a relational model $\mathcal{U}_{\text{para}}$. Since interpreted functions do not depend on local context, we sidestep issues with relational context highlighted by Altenkirch and others [1], thus result of the interpretation is a proper proof of a free theorem.

3.2.3 Synthetic mathematics. As thoroughly discussed in a recent paper by Nawrocki and others [10], thanks to MLTT being an internal language of certain $(\infty, 1)$-categories, we can do *synthetic mathematics* inside it: prove a theorem inside MLTT about types; interpret it in a model of MLTT and yield a proof about objects in that model. It is easy to see that reinterpretation operator we have in MLTT-R is sufficient to do synthetic mathematics in it and immediately apply results interpreted in a suitable model.

4 Related work

To the best of our knowledge, we didn't find any research which explores the topic at hand, but the idea is certainly in the air, as illustrated by the following brilliant papers.

Original work by Lambek [7] sets the tone of the future research, establishing correspondence between type systems based on lambda calculus with various kinds of categories, starting with correspondence between simply typed lambda calculus and cartesian closed categories.

Conal Elliott's "Compiling to Categories" [5] shows how to interpret (some subset of) Haskell in arbitrary `Category` (again, definable in Haskell) via GHC compiler plugin. That paper is a direct inspiration for this one. There is an important difference, though: reinterpretation is internal to our type systems, unlocking new patterns in the usage of reinterpretation, including (but not limited to) universe polymorphism, external parametricity and more. However, Conal's approach is more

$$\textsc{Infer-Uni}\ \frac{\Gamma\ \mathrm{wf}}{\Gamma \vdash \mathrm{U}_\ell \Rightarrow \mathcal{U}_{\ell+1}} \qquad \textsc{Infer-Nat}\ \frac{\Gamma\ \mathrm{wf}}{\Gamma \vdash \mathtt{nat}_{\mathcal{U}} \Rightarrow \mathcal{U}_0} \qquad \textsc{Check-Zero}\ \frac{\Gamma\ \mathrm{wf}}{\Gamma \vdash 0 \Leftarrow \mathrm{Nat}} \qquad \textsc{Check-Succ}\ \frac{\Gamma \vdash t \Leftarrow \mathrm{Nat}}{\Gamma \vdash \mathrm{S}\,t \Leftarrow \mathrm{Nat}}$$

$$\textsc{Infer-NatInd}\ \frac{\Gamma \vdash t \Rightarrow \mathrm{Nat} \qquad \Gamma, x : \mathrm{Nat} \vdash \mathrm{level}(T) = _ \qquad \Gamma \vdash s_0 \Leftarrow T[x \coloneqq 0] \qquad \Gamma, y : \mathrm{Nat} \vdash s_S \Leftarrow T[x \coloneqq \mathrm{S}\,y]}{\Gamma \vdash \mathrm{natind}_{x.T}\, t\, \{s_0, y \mapsto s_S\} \Rightarrow T[x \coloneqq t]}$$

$$\textsc{Infer-Sigma}\ \frac{\Gamma \vdash s \Rightarrow \mathcal{U}_\kappa \qquad \Gamma, x : \mathrm{El}_\kappa(s) \vdash t \Rightarrow \mathcal{U}_\ell}{\Gamma \vdash (x : s) \times_{\mathcal{U}} t \Rightarrow \mathcal{U}_{\max(\kappa,\ell)}} \qquad \textsc{Check-Pair}\ \frac{\Gamma \vdash s \Leftarrow S \qquad \Gamma \vdash t \Leftarrow T[x \coloneqq s]}{\Gamma \vdash (s, t) \Leftarrow (x : S) \times T}$$

$$\textsc{Infer-Proj}_1\ \frac{\Gamma \vdash t \Rightarrow (x : S) \times T}{\Gamma \vdash \pi_1\, t \Rightarrow S} \qquad \textsc{Infer-Proj}_2\ \frac{\Gamma \vdash t \Rightarrow (x : S) \times T}{\Gamma \vdash \pi_2\, t \Rightarrow T[x \coloneqq \pi_1\, t]} \qquad \textsc{Infer-Pi}\ \frac{\Gamma \vdash s \Rightarrow \mathcal{U}_\kappa \qquad \Gamma, x : \mathrm{El}_\kappa(s) \vdash t \Rightarrow \mathcal{U}_\ell}{\Gamma \vdash (x : s) \to_{\mathcal{U}} t \Rightarrow \mathcal{U}_{\max(\kappa,\ell)}}$$

$$\textsc{Check-Abs}\ \frac{\Gamma, x : S \vdash t \Leftarrow T}{\Gamma \vdash \lambda x.\, t \Leftarrow (x : S) \to T} \qquad \textsc{Infer-App}\ \frac{\Gamma \vdash t \Rightarrow (x : S) \to T \qquad \Gamma \vdash s \Leftarrow S}{\Gamma \vdash t\, s \Rightarrow T[x \coloneqq s]}$$

$$\textsc{Infer-Id}\ \frac{\Gamma \vdash t \Rightarrow \mathcal{U}_\ell \qquad \Gamma \vdash t_1, t_2 \Leftarrow \mathrm{El}_\ell(t)}{\Gamma \vdash \mathtt{id}\, t\, t_1\, t_2 \Rightarrow \mathcal{U}_\ell} \qquad \textsc{Check-Refl}\ \frac{\Gamma \vdash t_1 \equiv t_2 : T}{\Gamma \vdash \mathrm{refl} \Leftarrow t_1 =_T t_2}$$

$$\textsc{Infer-J}\ \frac{\Gamma \vdash t \Rightarrow t_1 =_T t_2 \qquad \Gamma, x : T, y : t_1 =_T x \vdash \mathrm{level}(S) = _ \qquad \Gamma \vdash s \Leftarrow S[x \coloneqq t_1, y \coloneqq \mathrm{refl}]}{\Gamma \vdash \mathrm{J}_{xy.S}\, t\, s \Rightarrow S[x \coloneqq t_2, y \coloneqq t]}$$

$$\textsc{Infer-Var}\ \frac{\text{🔒} \notin \Delta}{\Gamma, x : T, \Delta \vdash x \Rightarrow T} \qquad \textsc{Infer-Box}\ \frac{\Gamma.\text{🔒} \vdash t \Rightarrow \mathcal{U}_\ell}{\Gamma \vdash \Box_{\mathcal{U}} t \Rightarrow \mathcal{U}_\ell} \qquad \textsc{Check-Lock}\ \frac{\Gamma.\text{🔒} \vdash t \Leftarrow T}{\Gamma \vdash [t]_{\text{🔒}} \Leftarrow \Box T} \qquad \textsc{Infer-Unlock}\ \frac{\Gamma^{\text{🔓}} \vdash t \Rightarrow \Box T}{\Gamma \vdash [t]_{\text{🔓}} \Rightarrow T}$$

$$\textsc{Infer-Interp}\ \frac{\Gamma \vdash t \Rightarrow \Box T \qquad \Gamma \vdash \mathrm{level}(T) = \kappa}{\Gamma \vdash [\![t]\!]^{\ell_0,\ldots,\ell_\kappa} \Rightarrow \left(\mathtt{m} : \mathcal{M}_{\ell_0,\ldots,\ell_\kappa}\right) \to \mathtt{m}\,[\![T]\!]}$$

$$\textsc{Infer-Ascr}\ \frac{\Gamma \vdash \mathrm{level}(T) = _ \qquad \Gamma \vdash t \Leftarrow T}{\Gamma \vdash t : T \Rightarrow T} \qquad \textsc{Check-Inferred}\ \frac{\Gamma \vdash t \Rightarrow S \qquad \Gamma \vdash S \equiv T}{\Gamma \vdash t \Leftarrow T}$$

Fig. 7. Bidirectional term typing rules

flexible since his translation to categories requires desired category to only implement primitives actually used for translation of a particular term at hand.

In a recent paper "A Certifying Proof Assistant for Synthetic Mathematics in Lean", Wojciech Nawrocki and others [10] adapt Conal's approach to Lean's system with dependent types, using Categories-with-Families to represent internal models and employing Lean's powerful metaprogramming facilities instead of a compiler plugin. In doing so, they are able to provide a framework for doing synthetic mathematics in Lean, extracting proofs done in MLTT applied to more specific categories. Thus our results in section 3.2.3 are close to the results shown in their paper. To add to that, their work is already implemented in practical proof assistant. Yet, our type system involves no metaprogramming since reinterpretation is internal to it, and it handles conversion rule gracefully. In addition, the focus is a bit different, where we use reinterpretation to express other polymorphic features of a language.

Conor McBride's "Crude but effective stratification" [8] (systemic treatment of this idea is given by Favonia and others [6]) introduces an eye-opening idea of using *displacement* as a lightweight form of universe polymorphism: to reuse term typed at a lower universe level, lift all levels in a term by adding a constant shift s. From the point of view advocated by this paper, you can see it as an act of reinterpreting an expression in an internal model of type system where universe at level i is interpreted as a universe at level $i + s$ (section 3.2.1).

In "Parametricity and Dependent Types" by Jean-Philippe Bernardy, Patrik Jansson and Ross Paterson [2], authors introduce a direct translation procedure to yield parametricity theorems and proofs from polymorphic definitions in a dependently typed system. The definition of «external parametricity» we follow, though, is given in "Internal Parametricity, without an Interval" by Thorsten Altenkirch, Yorgo Chamoun, Ambrus Kaposi and Michael Shulman [1]. Once again, we can reformulate the translation they give as reinterpretation in an internal model. Altenkirch and others noted an issue with this approach: if we apply the external translation to the judgment with nonempty context, context becomes parametric, too; that's the reason they add `rel` operation and yield type system with *computational properties of higher observational type theory*. We sidestep the issue with context since reinterpretation is applied only to terms whose context is also «reinterpretable» and is not affected by the translation (section 3.2.2).

5 Conclusions and further work

In retrospect, we managed to define two self-interpreting type systems, STLC-R and MLTT-R, meaning that they have a notion of their own «internal models» which they then can use to reinterpret their own terms inside such models. In doing so, such type systems get new features which previously were defined as separate builtin quirks of a system. Basic metatheoretical properties are proven; for future work we have the following plans:

- Provide mechanized proofs of metatheoretical properties of systems;
- Provide implementation of MLTT-R in a practical proof assistant;
- Handle «partial» interpretation a-la Conal Elliott's "Compiling to Categories" [5];
- Make equality in MLTT-R more extensional, most probably by making type system closer to Homotopy type theory [13];
- Relax context constraints in reinterpretable terms.

References

[1] Thorsten Altenkirch, Yorgo Chamoun, Ambrus Kaposi, and Michael Shulman. 2024. Internal Parametricity, without an Interval. *Proc. ACM Program. Lang.* 8, POPL, Article 78 (Jan. 2024), 30 pages. doi:10.1145/3632920

[2] Jean-Philippe Bernardy, Patrik Jansson, and Ross Paterson. 2010. Parametricity and dependent types. In *Proceedings of the 15th ACM SIGPLAN International Conference on Functional Programming* (Baltimore, Maryland, USA) *(ICFP '10)*. Association for Computing Machinery, New York, NY, USA, 345–356. doi:10.1145/1863543.1863592

[3] Simon Castellan, Pierre Clairambault, and Peter Dybjer. 2020. Categories with Families: Unityped, Simply Typed, and Dependently Typed. arXiv:1904.00827 [cs.LO] https://arxiv.org/abs/1904.00827

[4] Dominique Clément, Thierry Despeyroux, Gilles Kahn, and Joëlle Despeyroux. 1986. A simple applicative language: mini-ML. In *Proceedings of the 1986 ACM Conference on LISP and Functional Programming* (Cambridge, Massachusetts, USA) *(LFP '86)*. Association for Computing Machinery, New York, NY, USA, 13–27. doi:10.1145/319838.319847

[5] Conal Elliott. 2017. Compiling to categories. *Proc. ACM Program. Lang.* 1, ICFP, Article 27 (Aug. 2017), 27 pages. doi:10.1145/3110271

[6] Kuen-Bang Hou (Favonia), Carlo Angiuli, and Reed Mullanix. 2023. An Order-Theoretic Analysis of Universe Polymorphism. *Proc. ACM Program. Lang.* 7, POPL, Article 57 (Jan. 2023), 27 pages. doi:10.1145/3571250

[7] Joachim Lambek. 1980. From λ-calculus to cartesian closed categories. In *To H. B. Curry: Essays on Combinatory Logic, Lambda-Calculus and Formalism*, J. P. Seldin and J. R. Hindley (Eds.). Academic Press, London, 375–402.

[8] Conor McBride. 2002. Crude but Effective Stratification. https://personal.cis.strath.ac.uk/conor.mcbride/Crude.pdf

[9] Eugenio Moggi. 1991. Notions of computation and monads. *Information and Computation* 93, 1 (1991), 55–92. doi:10.1016/0890-5401(91)90052-4 Selections from 1989 IEEE Symposium on Logic in Computer Science.

[10] Wojciech Nawrocki, Joseph Hua, Mario Carneiro, Yiming Xu, Spencer Woolfson, Shuge Rong, Sina Hazratpour, and Steve Awodey. 2026. A Certifying Proof Assistant for Synthetic Mathematics in Lean. In *Proceedings of the 15th ACM SIGPLAN International Conference on Certified Programs and Proofs* (Rennes, France) *(CPP '26)*. Association for Computing Machinery, New York, NY, USA, 88–103. doi:10.1145/3779031.3779087

[11] John C Reynolds. 1983. Types, abstraction and parametric polymorphism. In *Information Processing 83*, R. E. A. Mason (Ed.). North-Holland, 513–523.

[12] Gregor Richards. 2025. Proof of the Church-Rosser Theorem. https://student.cs.uwaterloo.ca/~cs442/W25/extras/c-r-thm-proof.pdf

[13] The Univalent Foundations Program. 2013. *Homotopy Type Theory: Univalent Foundations of Mathematics*. https://homotopytypetheory.org/book, Institute for Advanced Study.